

EDS Practical :04

Name: Harsha Narnaware

PRN: 202501120026

Department: Data Science

Division: DS1

Batch: DS12

4.1.1 Pandas - series creation and manipulation:

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and

odd numbers separately.

- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
```

```
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
```

```
series = pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating  
the mean
```

```
grouped = series.groupby(series % 2 == 0).mean()
```

```
# Display the mean of even and odd numbers with  
labels
```

```
grouped.index = ['Even' if is_even else 'Odd' for is_even  
in grouped.index]
```

```
print("Mean of even and odd numbers:")
```

```
print(grouped)
```

Output:

The screenshot displays a test runner interface. At the top, it shows performance metrics: Average time (0.007 s, 7.00 ms) and Maximum time (0.017 s, 17.00 ms). Below these, it indicates that 3 out of 3 shown test case(s) passed and 3 out of 3 hidden test case(s) passed. The main section is for 'Test case 1', which took 17 ms and has a 'Debug' button. It compares 'Expected output' and 'Actual output' for a list of numbers 1 to 10. The outputs are identical, showing the mean of even and odd numbers, and the dtype as float64. At the bottom, there are tabs for 'Terminal' and 'Test cases'.

Average time	Maximum time	Test Results
0.007 s 7.00 ms	0.017 s 17.00 ms	3 out of 3 shown test case(s) passed 3 out of 3 hidden test case(s) passed

Test case 1 (17 ms)	
Expected output	Actual output
1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers: 5.0	Mean of even and odd numbers: 5.0
Odd: 5.0	Odd: 5.0
Even: 6.0	Even: 6.0
dtype: float64	dtype: float64

4.1.2. Dictionary to dataframe:

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.

- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.
- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35],  
}
```

```
# Convert the dictionary to a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Adding a new row
```

```
new_name = input("New name: ")
```

```
new_age = int(input("New age: "))
df.loc[len(df)] = [new_name, new_age]
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)
```

Modifying a row

```
row_to_modify = int(input("Index of row to modify: "))
new_age_value = int(input("New age: "))
df.at[row_to_modify, "Age"] = new_age_value
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)
```

Deleting a row

```
row_to_delete = int(input("Index of row to delete: "))
df =
df.drop(index=row_to_delete).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)
```

Adding a new column

```
genders=input("Enter genders separated by space:").split()
```

```
df["Gender"] = genders
```

Display the DataFrame after adding a new column

```
print("After adding a new column:")
```

```
print(df)
```

Modifying a column

```
df["Name"]=df["Name"].str.upper()
```

Display the DataFrame after modifying a column

```
print("After modifying a column:")
```

```
print(df)
```

Deleting a column

```
df=df.drop(columns=["Age"])
```

Display the DataFrame after deleting a column

```
print("After deleting a column:")
```

```
print(df)
```

Output:

Average time
0.085 s
85.00 ms

Maximum time
0.118 s
118.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 1 out of 1 hidden test case(s) passed

✓ Test case 1 118 ms Debug

Expected output	Actual output
Original DataFrame:	Original DataFrame:
.....Name..Age	Name..Age
0...Alice...25	0...Alice...25
1.....Bob...30	1.....Bob...30
2..Charlie...35	2..Charlie...35
New name: Susan	New name: Susan

Terminal Test cases

4.1.3. Student Information:

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Code:

```
import pandas as pd
```



```
# Read the text file into a DataFrame
```

```
file = input()
```

```
data = pd.read_csv(file, sep="\s+", header=None,  
names=["Name", "Age", "Grade"])
```

```
# write your code here..
```

```
df=pd.DataFrame(data)
```

```
print("First five rows:")
```

```
print(df.head(5))
```

```
print("Average age:",df["Age"].mean().round(2))
```

```
print("Students with a grade up to B")
```

```
filtered_df=df[df['Grade'].isin(['A','B'])]
```

```
print(filtered_df)
```

Output:

Average time
0.027 s
27.00 ms

Maximum time
0.038 s
38.00 ms

✓ 1 out of 1 shown test case(s) passed
✓ 1 out of 1 hidden test case(s) passed

✓ Test case 1 38 ms Debug

Expected output

Actual output

studentdata.txt	studentdata.txt
First five rows:	First five rows:
...Name...Age...Grade	...Name...Age...Grade
0..John...25....A	0..John...25....A
1..Alin...22....B	1..Alin...22....B
2..Emma...24....A	2..Emma...24....A

Terminal Test cases

4.2.1. Month with the Highest Total Sales:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
df["Month"] = df["Date"].dt.to_period("M")
```

```
df["Total Sales"] = df["Quantity"] * df["Price"]
```

```
monthly_sales = df.groupby("Month")["Total  
Sales"].sum()
```

```
# Find the month with the highest total sales
```

```
best_month = monthly_sales.idxmax()
```

```
highest_sales = monthly_sales.max()
```

```
print(f"Best month: {best_month}")
```

```
print(f"Total sales: ${highest_sales:.2f}")
```

Output:

The screenshot displays a test runner interface. At the top, it shows performance metrics: Average time (0.018 s, 17.67 ms) and Maximum time (0.035 s, 35.00 ms). To the right, it indicates that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. Below this, 'Test case 1' is shown with a 35 ms execution time and a 'Debug' button. The test case details are presented in a table comparing 'Expected output' and 'Actual output'.

Expected output	Actual output
sales_data.csv	sales_data.csv
Best-month: -2025-01	Best-month: -2025-01
Total-sales: -\$1210.00	Total-sales: -\$1210.00

At the bottom, there are tabs for 'Terminal' and 'Test cases'.

4.2.2. Best Selling Product:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
product_sales = df.groupby("Product")["Quantity"].sum()
```

```
# Find the product with the highest total quantity sold
```

```
best_product = product_sales.idxmax()
```

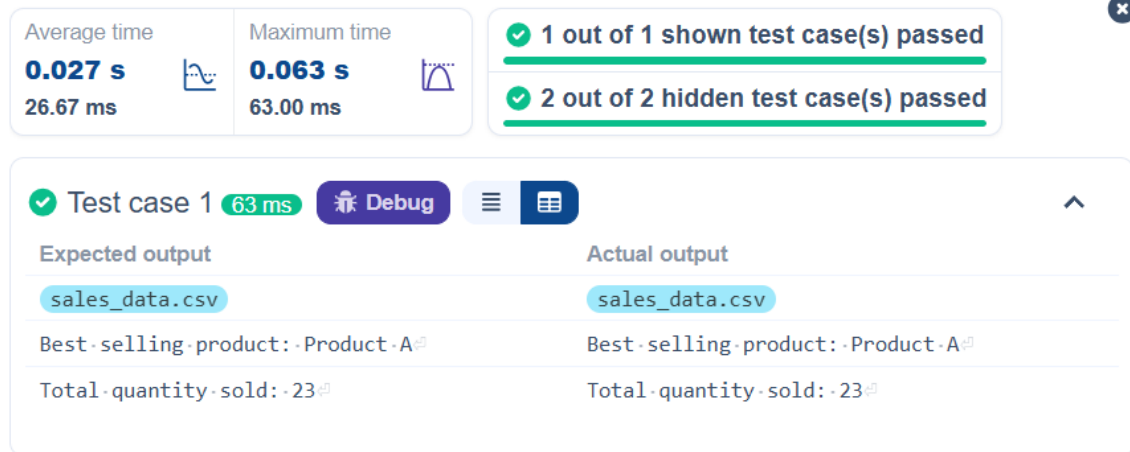
```
highest_quantity = product_sales.max()
```

```
# Display the result
```

```
print(f"Best selling product: {best_product}")
```

```
print(f"Total quantity sold: {highest_quantity}")
```

```
Output:
```



4.2.3. City that Sold the Most Products:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
city_sales = df.groupby("City")["Quantity"].sum()
```

```
best_city = city_sales.idxmax()
```

```
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```

Output:

The screenshot displays the execution results of a Python program. At the top, two boxes show performance metrics: 'Average time' is 0.019 s (18.67 ms) and 'Maximum time' is 0.038 s (38.00 ms). To the right, a green banner indicates '1 out of 1 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. Below this, a detailed view for 'Test case 1' (38 ms) is shown. It includes a 'Debug' button and a table comparing 'Expected output' and 'Actual output'. Both outputs are 'sales_data.csv' and 'City sold the most products: Los Angeles'.

Metric	Value
Average time	0.019 s (18.67 ms)
Maximum time	0.038 s (38.00 ms)

Test Results:

- 1 out of 1 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1 (38 ms):

Expected output	Actual output
sales_data.csv	sales_data.csv
City sold the most products: Los Angeles	City sold the most products: Los Angeles

4.2.4. Most Frequently Sold Product Pairs:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Code:

```
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

# write the code
date_products = {}

for date, group in df.groupby('Date'):
```



```
products = group['Product'].unique()
if len(products) > 1:
    date_products[date] = products
grouped = df.groupby("Date")["Product"].apply(list)
#GENERATE ALL PRODUCT PAIRS FOR EACH
CONSTRUCTIONS
pair_counter = Counter()

for products in date_products.values():
    pairs = combinations(sorted(products), 2)
    #sort to ensure consistency
    pair_counter.update(pairs)

    #find the most common products pairs
if pair_counter:
    max_count = max(pair_counter.values())

# Output the most frequent product pairs
for pair, count in pair_counter.items():
    if count == max_count:
        print(f"{pair[0]} and {pair[1]}: {count} times")
```

else:

```
print("No product pairs found.")
```

Output:

The screenshot displays a test runner interface. At the top, it shows performance metrics: Average time (0.020 s, 20.33 ms) and Maximum time (0.036 s, 36.00 ms). Below these, it indicates that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The main section shows 'Test case 1' with a '36 ms' duration and a 'Debug' button. It compares 'Expected output' and 'Actual output'. Both outputs are identical, showing 'sales_data.csv' followed by two lines of product pairs: 'Product A and Product B: 2 times' and 'Product A and Product C: 2 times'.

Average time	Maximum time	Test Results
0.020 s 20.33 ms	0.036 s 36.00 ms	✓ 1 out of 1 shown test case(s) passed ✓ 2 out of 2 hidden test case(s) passed

Test case 1 (36 ms) [Debug]	
Expected output	Actual output
sales_data.csv	sales_data.csv
Product A and Product B: 2 times	Product A and Product B: 2 times
Product A and Product C: 2 times	Product A and Product C: 2 times

4.2.5. Titanic Dataset Analysis and Data Cleaning:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation,

etc.) of the dataset using `.describe()`.

6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

1. Display the first 5 rows of the dataset

```
print(data.head())
```

2. Display the last 5 rows of the dataset

```
print(data.tail())
```

3. Get the shape of the dataset

```
print(data.shape)
```

4. Get a summary of the dataset (info)

```
print(data.info())
```

5. Get basic statistics of the dataset

```
print(data.describe())
```

6. Check for missing values

```
print(data.isnull().sum())
```

7. Fill missing values in the 'Age' column with the median age

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

8. Fill missing values in the 'Embarked' column with the mode

```
data['Embarked'].fillna(data['Embarked'].mode()[0],  
inplace=True)
```

9. Drop the 'Cabin' column due to many missing values

```
data.drop(columns=['Cabin'], inplace=True)
```

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

Output:

Test case 1 172 ms Debug

Expected output	Actual output
PassengerId Survived Pclass ... Fare Cabin Embarked	PassengerId Survived Pclass ... Fare Cabin Embarked
0 1 0 3 7.2500 NaN S	0 1 0 3 7.2500 NaN S
1 2 1 1 71.2833 C85 C	1 2 1 1 71.2833 C85 C
2 3 1 3 7.9250 NaN S	2 3 1 3 7.9250 NaN S
3 4 1 1 53.1000 C123 S	3 4 1 1 53.1000 C123 S
4 5 0 3 8.0500 NaN S	4 5 0 3 8.0500 NaN S
[5 rows x 12 columns]	[5 rows x 12 columns]
PassengerId Survived Pclass ... Fare Cabin Embarked	PassengerId Survived Pclass ... Fare Cabin Embarked
886 887 0 2 13.00 NaN S	886 887 0 2 13.00 NaN S
887 888 1 1 3 20.00 NaN S	887 888 1 1 3 20.00 NaN S

4.2.6. Titanic Dataset Analysis and Data Cleaning – 2:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.

5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
```

```
data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
# 3. One-hot encode the 'Embarked' column
```

```
data = pd.get_dummies(data, columns=['Embarked'])
```

```
# 4. Get the mean age of passengers
```

```
mean_age = data['Age'].mean()
```

```
print(mean_age)
```

```
# 5. Get the median fare of passengers
```

```
median_fare = data['Fare'].median()
```

```
print(median_fare)
```

```
# 6. Get the number of passengers by class
```

```
print( data['Pclass'].value_counts()
```

```
)
```

```
# 7. Get the number of passengers by gender
```

```
print( data['Sex'].value_counts())
```


8. Get the number of passengers by survival status

```
print( data['Survived'].value_counts())
```

9. Calculate the survival rate

```
survival_rate = data['Survived'].mean()
```

```
print(format(survival_rate))
```

10. Calculate the survival rate by gender

```
survival_by_gender = data.groupby('Sex')
```

```
['Survived'].mean()
```

```
print( survival_by_gender)
```

Output:

Average time

0.089 s

89.00 ms

Maximum time

0.089 s

89.00 ms

✓ 1 out of 1 shown test case(s) passed

✕

Dahinner

✓ Test case 1

89 ms

Debug

⋮

📄

^

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3...491	3...491
1...216	1...216
2...184	2...184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0...577	0...577
1...314	1...314
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0...549	0...549
1...342	1...342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0.3838383838383838	0.3838383838383838
Sex	Sex
0...0.188908	0...0.188908
1...0.742038	1...0.742038

4.2.7. Titanic Dataset Analysis and Data Cleaning – 3:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size

(FamilySize).

4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)  
data = pd.get_dummies(data, columns=['Embarked'],  
drop_first=True)
```

1. Calculate the survival rate by class

```
print(data.groupby('Pclass') ['Survived'].mean())
```

2. Calculate the survival rate by embarked location

```
print(data.groupby ('Embarked_S') ['Survived'].mean())
```

3. Calculate the survival rate by family size

```
print(data.groupby('FamilySize') ['Survived'].mean())
```

4. Calculate the survival rate by being alone

```
print(data.groupby('IsAlone') ['Survived'].mean())
```

5. Get the average fare by class

```
print(data.groupby('Pclass') ['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass') ['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1] ['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0] ['Pclass'].value_counts())
```

Output:

Average time
0.096 s
96.00 ms

Maximum time
0.096 s
96.00 ms

1 out of 1 shown test case(s) passed

Test case 1 96 ms

Debug

Expected output	Actual output
Pclass	Pclass
1...0.629630	1...0.629630
2...0.472826	2...0.472826
3...0.242363	3...0.242363
Name: -Survived, dtype: -float64	Name: -Survived, dtype: -float64
Embarked_S	Embarked_S
0...0.506073	0...0.506073
1...0.336957	1...0.336957
Name: -Survived, dtype: -float64	Name: -Survived, dtype: -float64
FamilySize	FamilySize
0...0.303538	0...0.303538
1...0.552795	1...0.552795

Terminal

Test cases

4.2.8. Titanic Dataset Analysis and Data Cleaning – 4:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'],
drop_first=True)
```

1. Get the number of survivors by gender

```
survivors_by_gender = data[data['Survived'] == 1]  
['Sex'].value_counts()
```

```
print(survivors_by_gender)
```

2. Get the number of non-survivors by gender

```
non_survivors_by_gender = data[data['Survived'] == 0]  
['Sex'].value_counts()
```

```
print(non_survivors_by_gender)
```

3. Get the number of survivors by embarked location

```
survivors_by_embarked_s = data[data['Survived'] == 1]  
['Embarked_S'].value_counts()
```

```
print(survivors_by_embarked_s)
```

4. Get the number of non-survivors by embarked location

```
non_survivors_by_embarked_s = data[data['Survived']  
== 0]['Embarked_S'].value_counts()
```

```
print(non_survivors_by_embarked_s)
```

5. Calculate the percentage of children (Age < 18) who survived

```
children = data[data['Age'] < 18]
```

```
children_survival_rate = children['Survived'].mean()
```

```
print(children_survival_rate)
```


6. Calculate the percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]
adults_survival_rate = adults['Survived'].mean()
print(adults_survival_rate)
```

7. Get the median age of survivors

```
median_age_survivors = data[data['Survived'] == 1]
['Age'].median()
print(median_age_survivors)
```

8. Get the median age of non-survivors

```
median_age_non_survivors = data[data['Survived'] == 0]
['Age'].median()
print(median_age_non_survivors)
```

9. Get the median fare of survivors

```
median_fare_survivors = data[data['Survived'] == 1]
['Fare'].median()
print(median_fare_survivors)
```

10. Get the median fare of non-survivors

```
median_fare_non_survivors = data[data['Survived'] == 0]
['Fare'].median()
```

```
print(median_fare_non_survivors)
```

Output:

Average time
0.116 s
116.00 ms

Maximum time
0.116 s
116.00 ms

1 out of 1 shown test case(s) passed

Test case 1 116 ms Debug

Expected output	Actual output
female...233	female...233
male.....109	male.....109
Name: Sex, dtype: int64	Name: Sex, dtype: int64
male.....468	male.....468
female....81	female....81
Name: Sex, dtype: int64	Name: Sex, dtype: int64
1...217	1...217
0...125	0...125
Name: Embarked_S, dtype: int64	Name: Embarked_S, dtype: int64
1...427	1...427
0...122	0...122
Name: Embarked_S, dtype: int64	Name: Embarked_S, dtype: int64
0.5398230088495575	0.5398230088495575
0.3810316139767055	0.3810316139767055
28.0	28.0
28.0	28.0